

Analytical Problem Solving

Warehouse Delivery Robot- solution

Assumption used in the search: whenever more than one move is possible, neighbours are considered in the order Up, Down, Left, Right. The same order is used for both BFS and DFS. A state is marked visited when it is added to the frontier, so it is not added again. This makes the step-by-step traces unambiguous.

The robot starts at (1,1), the package must be delivered at (5,5), and the blocked cells are (2,2), (2,3), (3,3), (4,2), and (4,4). Each legal move costs 1.

a. PEAS description

PEAS stands for Performance measure, Environment, Actuators, and Sensors. For this robot, the description is:

Performance measure

- Deliver the package successfully to cell (5,5).
- Avoid shelves, walls, and any collision.
- Use as few moves as possible because every move has a cost of 1.
- Complete the task reliably without entering an invalid cell.

Environment

The environment is a fixed 5×5 warehouse grid. It contains free floor cells, blocked shelving cells, a known starting position, and a known delivery point. Only horizontal and vertical movement is allowed. For this problem the map is static, discrete, fully observable, deterministic, sequential, and single-agent.

Actuators

The robot uses its drive system to execute one of four commands: Move Up, Move Down, Move Left, or Move Right. It may also stop when it reaches the delivery cell.

Sensors

The robot needs a position sensor (or localisation system) to know its current cell, obstacle or proximity sensors to detect shelves and walls, and feedback from its motion system to confirm that a commanded move was completed.

b. Most suitable type of agent

A goal-based agent is the best choice for this task. The robot is not merely reacting to the cell directly in front of it; it has a clear goal, namely reaching (5,5), and it must examine possible sequences of moves before choosing a route. A goal-based agent can represent the warehouse map, predict the result of each action, and use a search algorithm such as BFS to plan a path to the destination.

A simple reflex agent would be too limited because a rule such as “move forward if the next cell is free” can lead it into a dead end. A model-based agent would help the robot remember the map and its own location, but the model alone does not tell it which destination to reach. A utility-based agent could be useful in a more realistic warehouse where time, battery level, congestion, or risk must be balanced. Here, however, all moves have the same cost and the only stated objective is to reach one goal safely, so goal-based reasoning is sufficient and most appropriate.

c. Formal search problem

- Initial state: (1,1).
- Goal state: (5,5). The goal test is true when the current state equals (5,5).
- State space: all unblocked cells (r,c), where r and c are between 1 and 5. There are 25 cells in total and 5 are blocked, so there are 20 valid states.
- Actions: Up, Down, Left, and Right, provided the resulting cell remains inside the grid and is not blocked.
- Transition model: $\text{Result}((r,c), \text{Up}) = (r-1,c)$, $\text{Result}((r,c), \text{Down}) = (r+1,c)$, $\text{Result}((r,c), \text{Left}) = (r,c-1)$, and $\text{Result}((r,c), \text{Right}) = (r,c+1)$, whenever the destination is legal.
- Step cost: $c(s,a,s') = 1$ for every legal move.
- Path cost: the sum of all step costs. Therefore, a path containing n moves has total cost n.

d. Breadth-First Search (BFS)

BFS uses a FIFO queue. It expands all states at depth 0, then depth 1, then depth 2, and so on. In the table, the leftmost state in the frontier is the next one to be expanded.

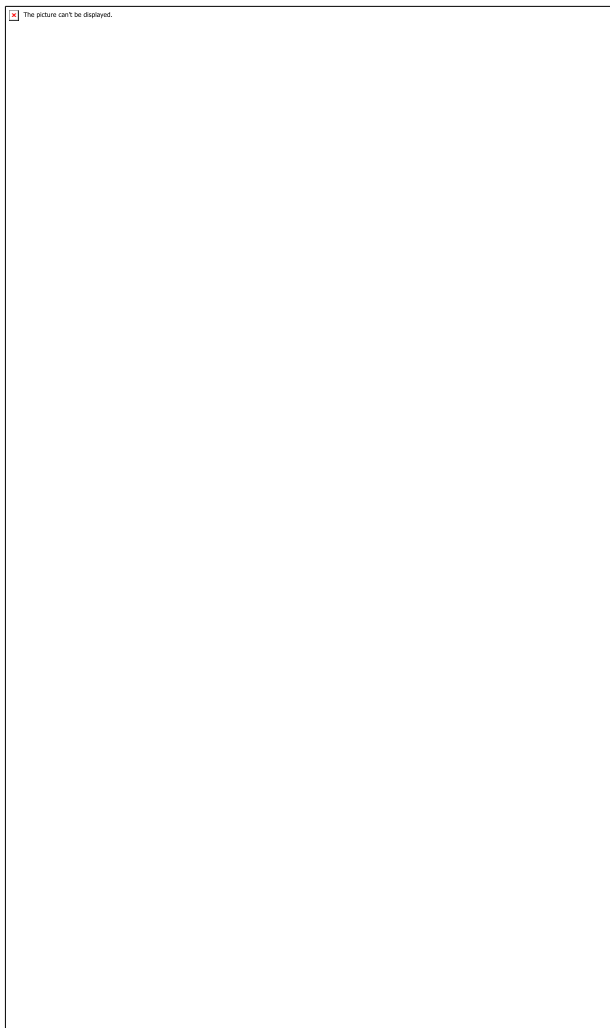
Step	Expanded cell	Queue (front first)
1	(1,1)	(2,1), (1,2)
2	(2,1)	(1,2), (3,1)
3	(1,2)	(3,1), (1,3)
4	(3,1)	(1,3), (4,1), (3,2)
5	(1,3)	(4,1), (3,2), (1,4)
6	(4,1)	(3,2), (1,4), (5,1)
7	(3,2)	(1,4), (5,1)
8	(1,4)	(5,1), (2,4), (1,5)
9	(5,1)	(2,4), (1,5), (5,2)
10	(2,4)	(1,5), (5,2), (3,4), (2,5)
11	(1,5)	(5,2), (3,4), (2,5)
12	(5,2)	(3,4), (2,5), (5,3)
13	(3,4)	(2,5), (5,3), (3,5)
14	(2,5)	(5,3), (3,5)
15	(5,3)	(3,5), (4,3), (5,4)
16	(3,5)	(4,3), (5,4), (4,5)
17	(4,3)	(5,4), (4,5)
18	(5,4)	(4,5), (5,5)
19	(4,5)	(5,5)
20	(5,5)	— — Goal reached.

Final BFS path

$(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)$

This route contains 8 moves, so its total cost is 8. Since BFS explores nodes in increasing depth and every move has equal cost, this is a shortest path.

Python program



e. Depth-First Search (DFS)

DFS uses a LIFO stack and follows one branch as deeply as possible before backtracking. The top of the stack is shown first in the frontier column. To preserve the stated Up, Down, Left, Right expansion order, available neighbours are pushed onto the stack in reverse order.

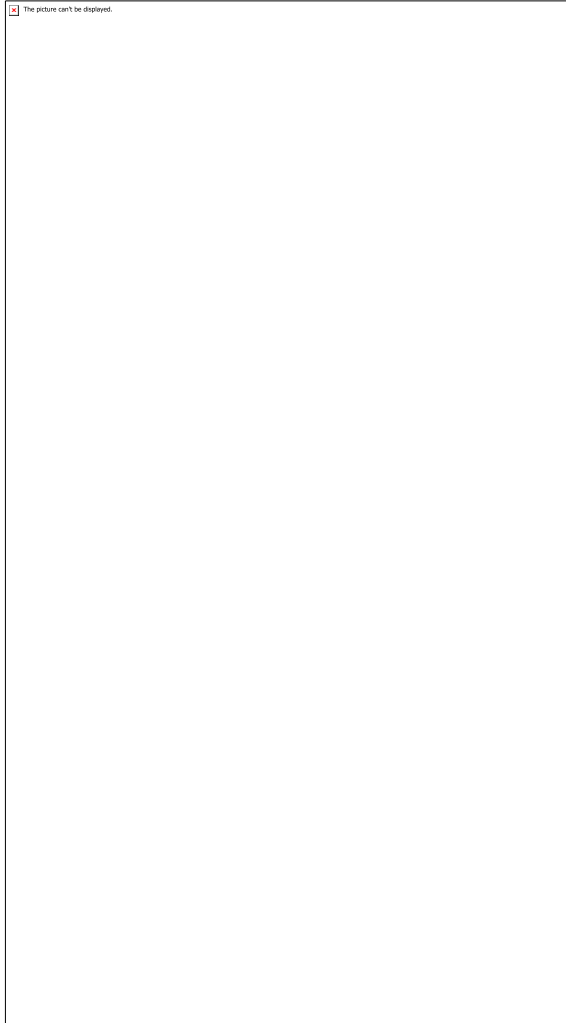
Step	Expanded cell	Stack (top first)
1	(1,1)	(2,1), (1,2)
2	(2,1)	(3,1), (1,2)
3	(3,1)	(4,1), (3,2), (1,2)
4	(4,1)	(5,1), (3,2), (1,2)
5	(5,1)	(5,2), (3,2), (1,2)
6	(5,2)	(5,3), (3,2), (1,2)
7	(5,3)	(4,3), (5,4), (3,2), (1,2)
8	(4,3)	(5,4), (3,2), (1,2)
9	(5,4)	(5,5), (3,2), (1,2)
10	(5,5)	(3,2), (1,2) — Goal reached.

Final DFS path

(1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5)

This route also contains 8 moves, so its total cost is 8. DFS first enters (4,3), which is a dead end because its other neighbouring cells are blocked or already visited. It then backtracks to (5,3), continues through (5,4), and reaches the goal. DFS happens to find a shortest route in this example, but that is a result of the chosen neighbour order and grid layout; DFS does not guarantee the shortest route in general.

Python program



f. Comparison of BFS and DFS

Criterion	BFS	DFS
Completeness	Complete on this finite grid. If a path exists, BFS will find one.	Complete here when a visited set is used because the state space is finite. Without cycle checking, DFS could revisit states indefinitely.
Optimality	Optimal because all step costs are 1. The first goal found is at the shallowest depth.	Not generally optimal. Its answer depends heavily on neighbour order. It happens to return cost 8 in this trace.
Time complexity	$O(b^d)$, where b is the branching factor and d is the depth of the shallowest goal.	$O(b^m)$, where m is the maximum search depth. On this finite graph, both are also bounded by $O(V+E)$ with duplicate checking.
Space complexity	$O(b^d)$, because it keeps the whole search frontier. In graph terms, storage is $O(V)$.	$O(bm)$, normally much smaller because it mainly stores the current branch plus alternatives. In graph terms, storage is $O(V)$.
Observed here	Expanded 20 states before removing the goal from the queue; returned a route of cost 8.	Expanded 10 states with the chosen order; returned a route of cost 8 after one dead-end branch.

Conclusion

BFS is the more suitable algorithm for this warehouse delivery problem because the robot should find a shortest route and every move has the same cost. Its main disadvantage is memory use, but that is insignificant for a grid with only 20 valid cells. DFS used fewer expansions in this particular run, yet that advantage is accidental: a different action order could send it through a long unhelpful branch. Therefore, BFS provides the stronger and more dependable choice for safe, cost-efficient delivery.